

PROJECT DOCUMENTATION

Bank Customer Churn Analysis

A Python & Power BI Data Analytics Project

Prepared By

Ashmita Karmakar

BCA (Bachelor of Computer Applications) Student

Tools Used: Python (Pandas, NumPy, Matplotlib, Seaborn) · Jupyter Notebook · Microsoft Power BI

September 2026

Abstract

Customer attrition, commonly referred to as churn, is one of the most pressing challenges faced by retail banks, as retaining an existing customer is significantly more cost-effective than acquiring a new one. This project presents an end-to-end data analytics workflow that investigates churn behaviour among 10,000 bank customers using a combination of Python and Microsoft Power BI. The Python component of the project focuses on loading, inspecting, cleaning, and validating a 21-column raw dataset, followed by the engineering of seven additional analytical features such as age groups, credit-score bands, balance categories, income categories, and customer activity levels. An extensive Exploratory Data Analysis (EDA) is then carried out, examining churn behaviour across demographic attributes (age, gender, state, city), financial attributes (account balance, income, credit score, loan status), behavioural attributes (transaction frequency, mobile banking and UPI usage, complaints) and account attributes (account type, tenure, number of products).

The cleaned and enriched dataset is subsequently loaded into Power BI, where a four-page interactive dashboard was designed covering an executive Overview, a dedicated Churn Drivers page, a Customer Segmentation page, and a searchable Customer Details page. The analysis reveals an overall churn rate of 18.38%, with the number of complaints emerging as the strongest single correlate of churn, while the number of banking products held and digital-channel usage (mobile banking and UPI) show a protective, churn-reducing relationship. The document that follows walks through every stage of the project — from raw data to cleaned data, from static charts to an interactive dashboard — and concludes with business insights, the challenges encountered, and recommendations for extending the project with predictive machine-learning models.

Table of Contents

Abstract	2
Table of Contents	3
1. Introduction.....	5
2. Problem Statement	5
3. Objectives	5
4. Dataset Description.....	6
4.1 Original Columns (21).....	6
4.2 Engineered Columns (Added During Feature Engineering)	7
5. Tools & Technologies	7
6. Python Workflow	9
6.1 Importing Libraries and Loading the Dataset.....	9
6.2 Initial Data Inspection	9
6.3 Missing Value and Duplicate Checks	10
6.4 Checking Unique Values and Category Frequencies	11
6.5 Data Cleaning	11
6.5.1 Removing Leading/Trailing Whitespace.....	12
6.5.2 Standardising Text Case	12
6.5.3 Converting the Date Column	12
6.6 Data Validation.....	12
6.6.1 Range Checks on Numerical Fields	13
6.6.2 Logical Consistency Checks	13
6.7 Feature Engineering	14
6.8 Data Quality Report and Export	16
7. Exploratory Data Analysis (EDA).....	17
7.1 Overall Churn Distribution	17
7.1.1 Churn Count and Overall Rate	17
7.1.2 Overall Churn Rate	17
7.2 Churn by Demographic Attributes.....	18
7.2.1 Churn Rate by Gender	18
7.2.2 Churn Rate by Age Group.....	18
7.2.3 Churn Rate by State	19
7.2.4 Top 10 Cities by Churn Rate	20
7.3 Churn by Account and Financial Attributes.....	21
7.3.1 Churn Rate by Account Type.....	21
7.3.2 Churn Rate by Tenure	22
7.3.3 Churn Rate by Number of Products.....	22
7.3.4 Churn Rate by Credit Card Ownership.....	23

7.3.5 Churn Rate by Loan Status	24
7.3.6 Churn Rate by Credit Score Range and Group	25
7.3.7 Churn Rate by Monthly Income (Range and Category)	25
7.3.8 Churn Rate by Account Balance Category	26
7.4 Churn by Behavioural Attributes	27
7.4.1 Churn Rate by Monthly Transactions	27
7.4.2 Churn Rate by Mobile Banking Usage	28
7.4.3 Churn Rate by UPI Usage	28
7.4.4 Churn Rate by Transaction Recency and Activity Category	29
7.4.5 Churn Rate by Number of Complaints.....	30
7.5 Stayed vs. Churned — Summary Comparison.....	31
7.5.1 Average Attribute Values by Churn Status.....	31
7.6 Correlation Analysis	31
7.6.1 Correlation Matrix.....	31
7.6.2 Correlation Heatmap.....	31
7.6.3 Variables Most Associated with Churn	32
7.7 EDA Summary	33
7.7.1 Final Summary Statistics	33
7.7.2 Finalising the Dataset for Power BI	34
8. Power BI Dashboard.....	36
8.1 Overview Page.....	36
8.2 Churn Drivers Page	36
8.3 Customer Segmentation Page	37
8.4 Customer Details Page	37
8.5 Dashboard Design Notes.....	38
9. Business Insights.....	38
9.1 Overall Takeaway	39
10. Challenges Faced	39
11. Future Improvements	40
12. Conclusion	41
13. References	41

1. Introduction

In the retail banking sector, customer churn — the phenomenon in which a customer closes their account, stops using their products, or moves their business to a competitor — has a direct and measurable impact on revenue, profitability, and long-term growth. Studies across the financial services industry consistently show that acquiring a new customer cost several times more than retaining an existing one, which means that even a small reduction in churn rate can translate into a meaningful improvement in profitability.

This project was undertaken as a personal, portfolio-oriented data analytics exercise to demonstrate a complete, real-world analytics pipeline: starting from a raw, uncleaned transactional dataset, moving through rigorous data cleaning and validation in Python, engineering new analytical features, performing structured exploratory data analysis, and finally translating the findings into an interactive, decision-support dashboard built in Microsoft Power BI. The project intentionally mirrors the workflow followed by data analysts in a real banking or fintech environment — Jupyter Notebook for exploration and preparation, and Power BI for stakeholder-facing reporting.

The dataset used contains 10,000 synthetic bank customer records spanning 21 raw attributes, covering demographic details (age, gender, state, city), account information (account type, tenure, balance, number of products), credit and loan details (credit score, loan status, loan amount), digital engagement (mobile banking, UPI usage, monthly transactions), and service-quality indicators (number of complaints), together with the binary churn label that this analysis centres around.

2. Problem Statement

Banks accumulate large volumes of customer data but often struggle to convert that data into clear, actionable insight about why customers leave. Without a structured analysis, it is difficult for decision-makers to know which customer segments are most at risk, which factors are most strongly associated with attrition, and where retention efforts should be prioritised.

The core problem this project addresses can be stated as follows: given a raw dataset of bank customer records containing demographic, financial, and behavioural attributes, how can churn patterns be identified, quantified, and communicated in a way that is both statistically grounded and easy for non-technical stakeholders to act upon? The project answers this question in two complementary stages — a rigorous, code-based Python analysis for data preparation and pattern discovery, and a visual, interactive Power BI dashboard for ongoing monitoring and stakeholder communication.

3. Objectives

The specific objectives of this project are:

- Load and thoroughly inspect the raw bank customer dataset (10,000 rows, 21 columns) to understand its structure, data types, and value ranges.
- Clean the dataset by removing whitespace inconsistencies, standardising categorical text case, correcting data types, and parsing date fields.

- Validate the dataset for missing values, duplicate records, out-of-range numerical values, and logically inconsistent combinations of fields (for example, a loan status of “No Loan” paired with a non-zero loan amount).
- Engineer new analytical features — Churn Status, Age Group, Credit Score Group, Balance Category, Income Category, Days Since Last Transaction, and Activity Category — to enable richer segmentation.
- Conduct exploratory data analysis (EDA) to quantify and visualise the relationship between churn and every major demographic, financial, and behavioural attribute in the dataset.
- Compute a correlation matrix to identify which numerical variables are most strongly associated with churn.
- Design and build an interactive, multi-page Power BI dashboard that allows stakeholders to explore churn drivers, customer segments, and individual customer profiles.
- Translate the statistical findings into clear business insights and actionable recommendations for customer retention.
- Document the entire workflow in a single, portfolio-ready report suitable for GitHub, LinkedIn, and job interviews.

4. Dataset Description

The raw dataset, `bank_customer_churn_analysis.csv`, contains 10,000 customer records and 21 columns spanning identity, demographic, account, credit, loan, and behavioural information, together with the binary Churn target. The table below documents all 21 original columns exactly as they appear in the source file.

4.1 Original Columns (21)

#	Column Name	Data Type	Description
1	Customer_ID	Text	Unique identifier assigned to each customer (e.g. CUST100001).
2	Full_Name	Text	Customer's full name.
3	Age	Integer	Customer age in years (observed range: 18 – 75).
4	Gender	Text	Customer gender — Male or Female.
5	State	Text	Indian state of residence (20 distinct states).
6	City	Text	City of residence (90 distinct cities).
7	Account_Type	Text	Type of bank account — Savings, Premium, Current, or Salary.
8	Tenure_Years	Integer	Number of years the customer has held an account with the bank (1 – 15).
9	Account_Balance	Integer	Current account balance in INR.
10	Num_Products	Integer	Number of banking products held by the customer (1 – 4).
11	Has_Credit_Card	Text	Whether the customer holds a credit card — Yes or No.
12	Loan_Status	Text	Loan status of the customer — Active, Closed, or No Loan.
13	Credit_Score	Integer	Customer's credit score (observed range: 420 – 850).

#	Column Name	Data Type	Description
14	Monthly_Income	Integer	Customer's monthly income in INR.
15	Loan_Amount	Integer	Outstanding loan amount in INR (0 when the customer has no loan).
16	Transactions_Per_Month	Integer	Average number of banking transactions per month (7 – 49).
17	Last_Transaction_Date	Date	Date of the customer's most recent transaction.
18	Mobile_Banking	Text	Whether the customer uses mobile banking — Yes or No.
19	UPI_Usage	Text	Whether the customer uses UPI (Unified Payments Interface) — Yes or No.
20	Complaints	Integer	Number of complaints filed by the customer (0 – 6).
21	Churn	Integer (0/1)	Target variable — 1 if the customer churned, 0 if the customer stayed.

4.2 Engineered Columns (Added During Feature Engineering)

In addition to the 21 raw columns, seven derived columns were engineered during the Python workflow (Section 6.6) to support richer, category-based analysis. After feature engineering the cleaned dataset therefore contains 28 columns in total.

#	Column Name	Type	Description
1	Churn_Status	Text	Human-readable label derived from Churn — “Stayed” or “Churned”.
2	Age_Group	Category	Age bucketed into 18-25, 26-35, 36-45, 46-55, and 56+.
3	Credit_Score_Group	Category	Credit score bucketed into Poor, Fair, Good, Very Good, and Excellent.
4	Balance_Category	Category	Account balance bucketed into No Balance, Low, Medium, High, and Very High.
5	Income_Category	Category	Monthly income split into four equal-sized quartile bands — Low, Medium, High, Very High.
6	Days_Since_Last_Transaction	Integer	Number of days between the analysis date and the customer's last transaction.
7	Activity_Category	Category	Recency of activity bucketed into Active, Recently Inactive, Inactive, and Highly Inactive.

5. Tools & Technologies

The project uses a standard, industry-representative data-analytics stack. Python was used inside Jupyter Notebook for all data preparation and exploratory analysis, and Microsoft Power BI Desktop was used to build the final interactive dashboard.

- Python 3 — core programming language for data processing and analysis.
- Pandas — data loading, cleaning, transformation, and aggregation.
- NumPy — numerical operations and array-based computations.
- Matplotlib & Seaborn — static statistical visualisations produced during EDA.

- Jupyter Notebook — interactive development environment for the Python workflow.
- Microsoft Power BI Desktop — interactive dashboard design, DAX measures, and data modelling.
- CSV — the interchange format used to move the cleaned dataset from Python into Power BI.

6. Python Workflow

This section documents the full Python workflow exactly as implemented in the accompanying Jupyter Notebook, `Bank_Customer_Churn_Analysis.ipynb`. Each subsection reproduces the actual code cells and their actual console/output, so that the logic and the results can be verified side by side.

6.1 Importing Libraries and Loading the Dataset

The workflow begins by importing the core data-analysis libraries and loading the raw CSV file into a Pandas DataFrame.

Python — Jupyter Notebook

```
#Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("All libraries imported successfully!")
```

Output

```
All libraries imported successfully!
```

Python — Jupyter Notebook

```
df = pd.read_csv("bank_customer_churn_analysis.csv")
```

6.2 Initial Data Inspection

Before any cleaning is performed, the dataset is inspected to understand its shape, column names, data types, and summary statistics.

Python — Jupyter Notebook

```
df.head()
```

Output

	Customer_ID	Full_Name	Age	Gender	State	City	\
0	CUST100001	Varun Mishra	29	Male	Delhi	New Delhi	
1	CUST100002	Ashish Menon	44	Male	Assam	Guwahati	
2	CUST100003	Sanjay Sarkar	52	Male	Bihar	Muzaffarpur	
3	CUST100004	Bhavna Shah	25	Female	Rajasthan	Jodhpur	
4	CUST100005	Sameer Sharma	57	Male	Maharashtra	Pune	

	Account_Type	Tenure_Years	Account_Balance	Num_Products	...	Loan_Status	\
0	Savings	4	76100	1	...	Active	
1	Premium	2	86600	3	...	Active	
2	Savings	9	45400	1	...	Closed	
3	Current	4	51900	1	...	No Loan	
4	Savings	6	118400	1	...	No Loan	

	Credit_Score	Monthly_Income	Loan_Amount	Transactions_Per_Month	\
0	625	68000	98000	22	
1	739	20700	418000	16	
2	667	31800	112000	22	
3	696	53500	0	29	
4	713	25500	0	22	

	Last_Transaction_Date	Mobile_Banking	UPI_Usage	Complaints	Churn
0	19-09-2026	No	Yes	0	0
1	11-06-2026	Yes	No	1	0
2	11-09-2026	Yes	Yes	1	0
3	06-07-2026	Yes	Yes	1	0
4	25-02-2026	Yes	Yes	0	1

[5 rows x 21 columns]

Python — Jupyter Notebook

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
df.columns
```

Output

```
Number of rows: 10000 | Number of columns: 21
Index(['Customer_ID', 'Full Name', 'Age', 'Gender', 'State', 'City', 'Account_Type',
      'Tenure_Years', 'Account_Balance', 'Num_Products', 'Has_Credit_Card', 'Loan_Status',
      'Credit_Score', 'Monthly_Income', 'Loan_Amount', 'Transactions_Per_Month', 'Last_Transaction_Date',
      'Mobile_Banking', 'UPI_Usage', 'Complaints', 'Churn'], dtype='str')
```

Python — Jupyter Notebook

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer_ID                          10000 non-null  str
1   Full_Name                            10000 non-null  str
2   Age                                  10000 non-null  int64
3   Gender                              10000 non-null  str
4   State                                10000 non-null  str
5   City                                 10000 non-null  str
6   Account_Type                         10000 non-null  str
7   Tenure_Years                         10000 non-null  int64
8   Account_Balance                      10000 non-null  int64
9   Num_Products                         10000 non-null  int64
10  Has_Credit_Card                      10000 non-null  str
11  Loan_Status                          10000 non-null  str
12  Credit_Score                         10000 non-null  int64
13  Monthly_Income                       10000 non-null  int64
14  Loan_Amount                          10000 non-null  int64
15  Transactions_Per_Month                10000 non-null  int64
16  Last_Transaction_Date                 10000 non-null  str
17  Mobile_Banking                       10000 non-null  str
18  UPI_Usage                            10000 non-null  str
19  Complaints                           10000 non-null  int64
20  Churn                                10000 non-null  int64
dtypes: int64(10), str(11)
memory usage: 1.6 MB
```

Python — Jupyter Notebook

```
df.describe()
```

Output

	Age	Tenure_Years	Account_Balance	Num_Products	Credit_Score	\
count	10000.00000	10000.00000	1.000000e+04	10000.00000	10000.00000	
mean	38.16700	6.771700	1.287244e+05	1.731400	710.141600	
std	10.57558	3.815709	1.304017e+05	0.798572	65.070838	
min	18.00000	1.000000	3.900000e+03	1.000000	420.000000	
25%	31.00000	4.000000	5.090000e+04	1.000000	667.000000	
50%	38.00000	6.000000	9.005000e+04	2.000000	710.500000	
75%	45.00000	10.000000	1.589000e+05	2.000000	755.000000	
max	75.00000	15.000000	1.500000e+06	4.000000	850.000000	

	Monthly_Income	Loan_Amount	Transactions_Per_Month	Complaints	\
count	10000.00000	1.000000e+04	10000.00000	10000.00000	
mean	55846.71000	2.628641e+05	24.009800	0.703100	
std	28134.537074	3.841229e+05	4.922106	0.845826	
min	15000.00000	0.000000e+00	7.000000	0.000000	
25%	36200.00000	0.000000e+00	21.000000	0.000000	
50%	49900.00000	1.130000e+05	24.000000	1.000000	
75%	68600.00000	4.040000e+05	27.000000	1.000000	
max	300000.00000	3.000000e+06	49.000000	6.000000	

	Churn
count	10000.00000
mean	0.18380
std	0.38734
min	0.00000
25%	0.00000
50%	0.00000
75%	0.00000
max	1.00000

6.3 Missing Value and Duplicate Checks

The dataset is checked for missing (null) values and fully duplicated rows before proceeding further.

Python — Jupyter Notebook

```
#Check Missing Values
```

```
df.isnull().sum()
```

Output

```
Customer_ID      0
Full_Name        0
Age              0
Gender           0
State            0
City             0
Account_Type     0
Tenure_Years     0
Account_Balance  0
Num_Products     0
Has_Credit_Card  0
Loan_Status      0
Credit_Score     0
Monthly_Income   0
Loan_Amount      0
Transactions_Per_Month 0
Last_Transaction_Date 0
Mobile_Banking   0
UPI_Usage        0
Complaints       0
Churn            0
dtype: int64
```

Python — Jupyter Notebook

```
#Check Duplicate Rows
df.duplicated().sum()
```

Output

```
np.int64(0)
```

6.4 Checking Unique Values and Category Frequencies

The unique categories present in key text columns are inspected, along with their frequency distributions, to confirm that categorical fields are clean and well-formed before standardisation.

Python — Jupyter Notebook

```
df["Gender"].unique()
```

Output

```
<StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

Python — Jupyter Notebook

```
df["Account_Type"].unique()
```

Output

```
<StringArray>
['Savings', 'Premium', 'Current', 'Salary']
Length: 4, dtype: str
```

Python — Jupyter Notebook

```
df["Gender"].value_counts()

df["Account_Type"].value_counts()

df["Loan_Status"].value_counts()

df["Churn"].value_counts()

df["Churn"].value_counts(normalize=True) * 100
```

Output

```
Gender: Male 5460, Female 4540 | Account_Type: Savings 6172, Salary 1591, Current 1455, Premium
782 | Loan_Status: No Loan 4748, Active 3437, Closed 1815 | Churn: 0 -> 8162 (81.62%), 1 ->
1838 (18.38%)
```

6.5 Data Cleaning

Data cleaning addressed three concerns: stray leading/trailing whitespace in text fields, inconsistent capitalisation across categorical values, and the raw date column being stored as plain text rather than a proper date type.

6.5.1 Removing Leading/Trailing Whitespace

Python — Jupyter Notebook

```
#Check all text columns
text_columns = df.select_dtypes(include="str").columns

print("Text columns:")
print(text_columns.tolist())
```

Output

```
Text columns:
['Customer_ID', 'Full_Name', 'Gender', 'State', 'City', 'Account_Type', 'Has_Credit_Card',
'Loan_Status', 'Mobile_Banking', 'UPI_Usage']
```

Python — Jupyter Notebook

```
#Remove Leading/Trailing Spaces
for col in text_columns:
    df[col] = df[col].str.strip()
```

6.5.2 Standardising Text Case

Python — Jupyter Notebook

```
#Standardize Text Case
categorical_columns = [
    "Gender",
    "State",
    "City",
    "Account_Type",
    "Has_Credit_Card",
    "Loan_Status",
    "Mobile_Banking",
    "UPI_Usage"
]

for col in categorical_columns:
    df[col] = df[col].str.strip().str.title()
```

6.5.3 Converting the Date Column

Python — Jupyter Notebook

```
#Convert Date Column
df["Last_Transaction_Date"] = pd.to_datetime(
    df["Last_Transaction_Date"],
    errors="coerce"
)
df["Last_Transaction_Date"].dtype
```

Output

```
dtype('<M8[us]')
```

Python — Jupyter Notebook

```
#Check Invalid Dates
df["Last_Transaction_Date"].isnull().sum()
```

Output

```
np.int64(0)
```

6.6 Data Validation

Validation checks were carried out to confirm that every numerical field falls within a logical range, and that combinations of related fields are internally consistent. Every check below returned an empty result set, confirming that the underlying dataset required no corrective action beyond the cleaning already performed.

6.6.1 Range Checks on Numerical Fields

Python — Jupyter Notebook

```
#Check Numerical Columns
numeric_columns = [
    "Age",
    "Tenure_Years",
    "Account_Balance",
    "Num_Products",
    "Credit_Score",
    "Monthly_Income",
    "Loan_Amount",
    "Transactions_Per_Month",
    "Complaints",
    "Churn"
]

df[numeric_columns].dtypes
```

Output

```
Age                int64
Tenure_Years       int64
Account_Balance    int64
Num_Products       int64
Credit_Score       int64
Monthly_Income     int64
Loan_Amount        int64
Transactions_Per_Month int64
Complaints         int64
Churn              int64
dtype: object
```

Python — Jupyter Notebook

```
df["Age"].min(), df["Age"].max()
```

Output

```
(np.int64(18), np.int64(75))
```

Python — Jupyter Notebook

```
#Check Age
df[(df["Age"] < 18) | (df["Age"] > 100)]

df["Credit_Score"].min(), df["Credit_Score"].max()

#Check Credit Score
df[
    (df["Credit_Score"] < 300) |
    (df["Credit_Score"] > 850)
]

#Check Monthly Income
df[df["Monthly_Income"] <= 0]

#Check Account Balance
df[df["Account_Balance"] < 0]

#Check Number of Products
df["Num_Products"].unique()

#Check Transactions
df[df["Transactions_Per_Month"] < 0]

#Check Complaints
df[df["Complaints"] < 0]
```

Output

```
All range-check queries returned an empty DataFrame (0 rows): no invalid ages (outside 18-100), no out-of-range credit scores (outside 300-850), no non-positive incomes, no negative account balances, no negative transaction counts, and no negative complaint counts were found. Num_Products confirmed to contain only the valid values [1, 2, 3, 4].
```

6.6.2 Logical Consistency Checks

These checks confirm that related fields agree with one another — for example, that a customer with a loan status of “No Loan” never has a positive outstanding loan amount, and that binary Yes/No fields contain only the two expected values.

Python — Jupyter Notebook

```
df[
    (df["Loan_Status"] == "No Loan") &
    (df["Loan_Amount"] > 0)
]

df[
    (df["Loan_Status"] == "Active") &
    (df["Loan_Amount"] <= 0)
]

#Check Credit Card Consistency
df[
    ~df["Has_Credit_Card"].isin(["Yes", "No"])
]

#Check Mobile Banking
df[
    ~df["Mobile_Banking"].isin(["Yes", "No"])
]

#Check UPI Usage
df[
    ~df["UPI_Usage"].isin(["Yes", "No"])
]
```

Output

All logical-consistency queries returned an empty DataFrame (0 rows): no “No Loan” customer has a positive Loan_Amount, no “Active” loan has a zero/negative Loan_Amount, and Has_Credit_Card, Mobile_Banking and UPI_Usage each contain only the values “Yes”/“No”.

Python — Jupyter Notebook

```
#Check Customer ID Format
df["Customer_ID"].head(10)
```

Output

```
0    CUST100001
1    CUST100002
2    CUST100003
3    CUST100004
4    CUST100005
5    CUST100006
6    CUST100007
7    CUST100008
8    CUST100009
9    CUST100010
Name: Customer_ID, dtype: str
```

6.7 Feature Engineering

Seven new columns were engineered from the cleaned raw data to support richer, category-based exploratory analysis and Power BI reporting. Section 4.2 documents the purpose of each of these seven columns; the combined code that creates them is reproduced below, followed by the resulting value counts for each new feature.

Python — Jupyter Notebook

```
#Feature Engineering
df["Churn_Status"] = df["Churn"].map({
    0: "Stayed",
    1: "Churned"
})

df["Churn_Status"].value_counts()

#Create Age Groups
df["Age_Group"] = pd.cut(
    df["Age"],
    bins=[17, 25, 35, 45, 55, 100],
```

```

        labels=[
            "18-25",
            "26-35",
            "36-45",
            "46-55",
            "56+"
        ]
    )

df["Age_Group"].value_counts().sort_index()

#Create Credit Score Groups
df["Credit_Score_Group"] = pd.cut(
    df["Credit_Score"],
    bins=[299, 579, 669, 739, 799, 850],
    labels=[
        "Poor",
        "Fair",
        "Good",
        "Very Good",
        "Excellent"
    ]
)

df["Credit_Score_Group"].value_counts().sort_index()

#Create Balance Categories
df["Balance_Category"] = pd.cut(
    df["Account_Balance"],
    bins=[-1, 0, 50000, 100000, 250000, np.inf],
    labels=[
        "No Balance",
        "Low",
        "Medium",
        "High",
        "Very High"
    ]
)

df["Balance_Category"].value_counts().sort_index()

#Create Income Categories
df["Income_Category"] = pd.qcut(
    df["Monthly_Income"],
    q=4,
    labels=[
        "Low",
        "Medium",
        "High",
        "Very High"
    ]
)

df["Income_Category"].value_counts().sort_index()

#Create Recency / Inactivity Feature
analysis_date = pd.Timestamp("2026-09-19")

df["Days_Since_Last_Transaction"] = (
    analysis_date - df["Last_Transaction_Date"]
).dt.days

df["Days_Since_Last_Transaction"].describe()

#Create Activity Categories
df["Activity_Category"] = pd.cut(
    df["Days_Since_Last_Transaction"],
    bins=[-1, 30, 90, 180, np.inf],
    labels=[
        "Active",
        "Recently Inactive",
        "Inactive",
        "Highly Inactive"
    ]
)

df["Activity_Category"].value_counts().sort_index()

```

Output

```

Churn_Status – Stayed: 8,162, Churned: 1,838.
Age_Group – 18-25: 1,266, 26-35: 2,805, 36-45: 3,451, 46-55: 1,920, 56+: 558.
Credit_Score_Group – Poor: 251, Fair: 2,369, Good: 4,115, Very Good: 2,373, Excellent: 892.
Balance_Category – No Balance: 0, Low: 2,440, Medium: 3,041, High: 3,379, Very High: 1,140.
Income_Category (quartile) – Low: 2,513, Medium: 2,494, High: 2,497, Very High: 2,496.
Days_Since_Last_Transaction – mean 53.37 days, min 0, max 365 days (25th pct 15, median 37, 75th
pct 76).
Activity_Category – Active: 4,348, Recently Inactive: 3,783, Inactive: 1,533, Highly Inactive: 336.

```

6.8 Data Quality Report and Export

A final data-quality report was generated to confirm the cleaned dataset's integrity before it was exported to CSV for use in both the remaining EDA and the Power BI dashboard.

Python — Jupyter Notebook

```

print("DATA QUALITY REPORT : ")

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Duplicate rows:", df.duplicated().sum())
print("Duplicate Customer IDs:", df["Customer_ID"].duplicated().sum())
print("Missing values:", df.isnull().sum().sum())
print("Churned customers:", df["Churn"].sum())
print("Churn rate:", round(df["Churn"].mean() * 100, 2), "%")

```

Output

```

DATA QUALITY REPORT :
Rows: 10000
Columns: 28
Duplicate rows: 0
Duplicate Customer IDs: 0
Missing values: 0
Churned customers: 1838
Churn rate: 18.38 %

```

Python — Jupyter Notebook

```

df.to_csv(
    "bank_customer_churn_cleaned.csv",
    index=False
)

print("Cleaned dataset exported successfully.")

```

Output: “Cleaned dataset exported successfully.” The cleaned file, bank_customer_churn_cleaned.csv, was then used as the single source of truth for all subsequent exploratory analysis and dashboard development.

7. Exploratory Data Analysis (EDA)

With the cleaned and feature-engineered dataset in hand, a structured exploratory data analysis was carried out to quantify how churn varies across every demographic, financial, account-level, and behavioural attribute in the dataset. Each subsection below pairs the exact analysis code with its console output and the resulting chart, followed by a short interpretation of the pattern observed.

7.1 Overall Churn Distribution

7.1.1 Churn Count and Overall Rate

Python — Jupyter Notebook

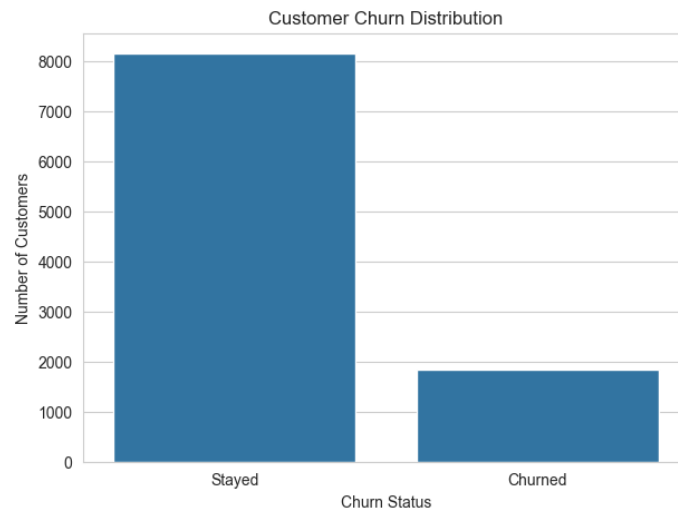
```
# Create a summary table for churn

churn_summary = pd.DataFrame({
    "Customer Status": ["Stayed", "Churned"],
    "Count": [
        (df["Churn"] == 0).sum(),
        (df["Churn"] == 1).sum()
    ],
    "Percentage": [
        round((df["Churn"] == 0).mean() * 100, 2),
        round((df["Churn"] == 1).mean() * 100, 2)
    ]
})

churn_summary
```

Output

	Customer Status	Count	Percentage
0	Stayed	8162	81.62
1	Churned	1838	18.38



Out of 10,000 customers, 8,162 (81.62%) stayed with the bank while 1,838 (18.38%) churned. An overall churn rate of roughly one in five customers gives a clear baseline against which every other segment in this report is compared — a segment with a churn rate meaningfully above 18.38% is a higher-risk group, and one meaningfully below it is a lower-risk group.

7.1.2 Overall Churn Rate

Python — Jupyter Notebook

```
# Calculate overall churn rate

churn_rate = df["Churn"].mean() * 100
```

```
print("Overall Churn Rate:", round(churn_rate, 2), "%")
```

Output

```
Overall Churn Rate: 18.38 %
```

7.2 Churn by Demographic Attributes

7.2.1 Churn Rate by Gender

Python — Jupyter Notebook

```
# Churn count by gender

pd.crosstab(
    df["Gender"],
    df["Churn"]
)

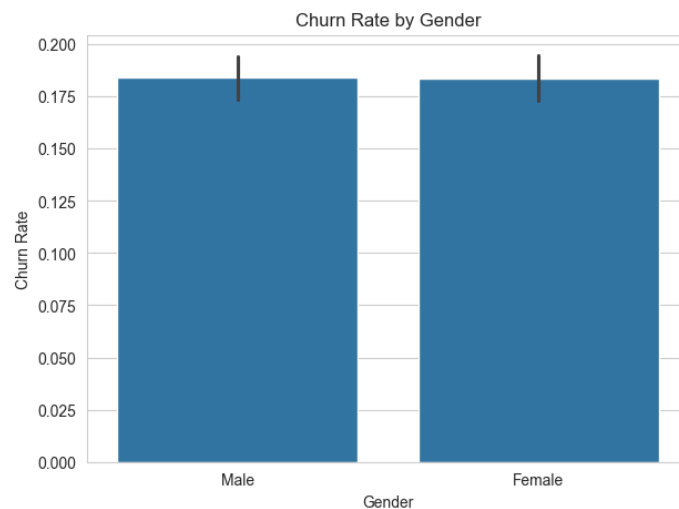
# Churn percentage by gender

gender_churn = pd.crosstab(
    df["Gender"],
    df["Churn"],
    normalize="index"
) * 100

gender_churn.round(2)
```

Output

Crosstab counts – Female: 3,706 stayed / 834 churned. Male: 4,456 stayed / 1,004 churned. As a percentage of each gender – Female: 81.63% stayed / 18.37% churned. Male: 81.61% stayed / 18.39% churned.



Churn is almost perfectly balanced by gender — 18.37% for female customers versus 18.39% for male customers. Gender shows no meaningful influence on churn in this dataset, indicating that retention strategies do not need to be differentiated by gender.

7.2.2 Churn Rate by Age Group

Python — Jupyter Notebook

```
# Churn count by age group

pd.crosstab(
    df["Age_Group"],
    df["Churn"]
)

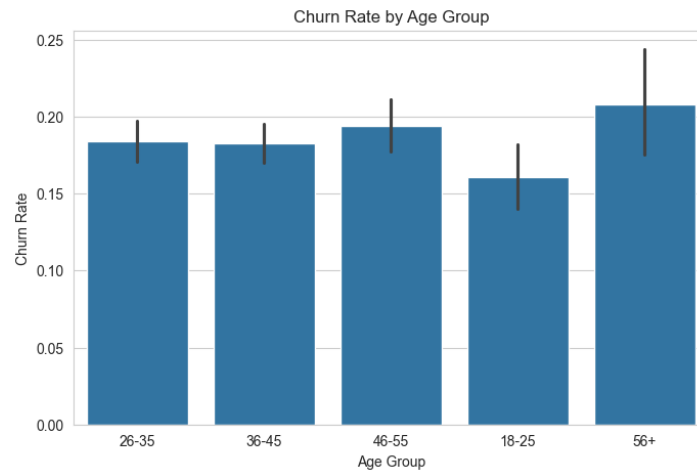
# Churn rate by age group
```

```
age_churn = pd.crosstab(
    df["Age_Group"],
    df["Churn"],
    normalize="index"
) * 100

age_churn.round(2)
```

Output

Counts (Stayed / Churned) — 18-25: 1,063 / 203. 26-35: 2,289 / 516. 36-45: 2,820 / 631. 46-55: 1,548 / 372. 56+: 442 / 116. Churn rate — 18-25: 16.03%, 26-35: 18.40%, 36-45: 18.28%, 46-55: 19.38%, 56+: 20.79%.



Churn rises steadily with age, from 16.03% in the 18-25 group to 20.79% in the 56+ group. Older customers appear modestly more likely to churn than younger customers, which may reflect changing financial needs or increased price/service sensitivity later in life.

7.2.3 Churn Rate by State

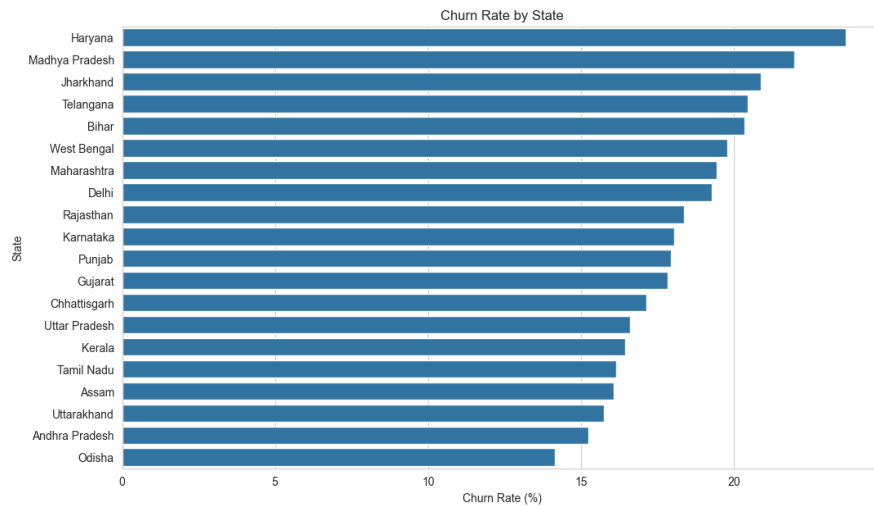
Python — Jupyter Notebook

```
# Calculate the churn rate for each state
state_churn = (
    df.groupby("State")["Churn"]
    .mean()
    .mul(100)
    .sort_values(ascending=False)
)

# Display the churn rate rounded to 2 decimal places
state_churn.round(2)
```

Output

```
State
Haryana                23.66
Madhya Pradesh         21.97
Jharkhand              20.87
Telangana              20.45
Bihar                 20.33
West Bengal            19.78
Maharashtra            19.42
Delhi                 19.28
Rajasthan              18.37
Karnataka              18.04
Punjab                17.93
Gujarat               17.84
Chhattisgarh          17.13
Uttar Pradesh          16.60
Kerala                16.46
Tamil Nadu            16.14
Assam                 16.06
Uttarakhand           15.75
Andhra Pradesh         15.24
Odisha                14.14
Name: Churn, dtype: float64
```



Churn varies noticeably by state, ranging from 23.66% in Haryana — the highest of the 20 states — down to 14.14% in Odisha, the lowest. Madhya Pradesh (21.97%), Jharkhand (20.87%), Telangana (20.45%) and Bihar (20.33%) round out the five highest-churn states, all sitting well above the 18.38% national average.

7.2.4 Top 10 Cities by Churn Rate

Python — Jupyter Notebook

```
# Calculate churn rate for each city
city_churn = (
    df.groupby("City")["Churn"]
    .mean()
    .mul(100)
    .sort_values(ascending=False)
)

# Display churn rates
city_churn.round(2)

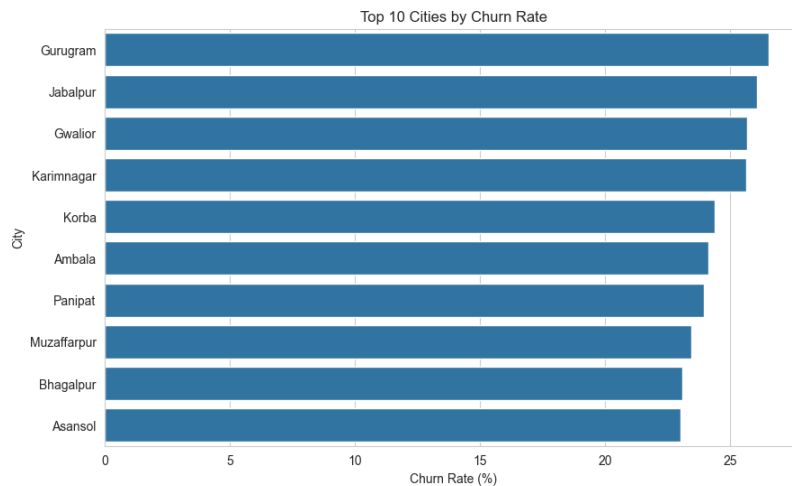
# GET TOP 10 CITIES BY CHURN RATE

# Select the 10 cities with the highest churn rates
top_10_cities = city_churn.head(10)

# Display the result
top_10_cities.round(2)
```

Output

Churn rate computed across all 90 cities (range: 6.67% – 26.56%). Top 10 highest-churn cities – Gurugram 26.56%, Jabalpur 26.09%, Gwalior 25.68%, Karimnagar 25.66%, Korba 24.39%, Ambala 24.14%, Panipat 23.94%, Muzaffarpur 23.46%, Bhagalpur 23.08%, Asansol 23.03%.



At the city level the spread is even wider: Gurugram tops the list at 26.56% churn, followed by Jabalpur (26.09%), Gwalior (25.68%), Karimnagar (25.66%) and Korba (24.39%). At the opposite end of the full 90-city list, Dehradun records the lowest churn rate at just 6.67%. This city-level granularity is valuable for the bank's regional retention teams, as it allows resources to be targeted at specific branches or service areas rather than entire states.

7.3 Churn by Account and Financial Attributes

7.3.1 Churn Rate by Account Type

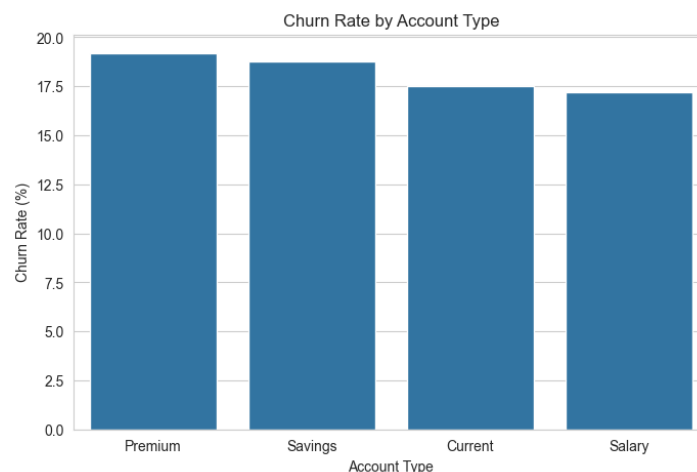
Python — Jupyter Notebook

```
# Calculate churn rate for each account type
account_churn = (
    df.groupby("Account_Type")["Churn"]
      .mean()
      .mul(100)
      .sort_values(ascending=False)
)

# Display the result
account_churn.round(2)
```

Output

```
Account_Type
Premium    19.18
Savings    18.78
Current    17.53
Salary     17.22
Name: Churn, dtype: float64
```



Premium accounts show the highest churn rate (19.18%), followed by Savings (18.78%), Current (17.53%) and Salary accounts, which are the most stable at 17.22%. Somewhat counter-intuitively, Premium account holders — typically a bank's most valuable segment — churn slightly more than average, making them a priority group for proactive relationship management.

7.3.2 Churn Rate by Tenure

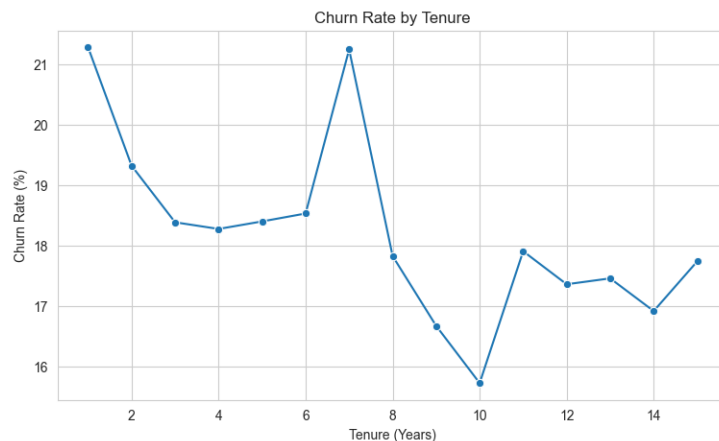
Python — Jupyter Notebook

```
# Calculate churn rate for each tenure value
tenure_churn = (
    df.groupby("Tenure_Years")["Churn"]
      .mean()
      .mul(100)
)

# Display the result
tenure_churn.round(2)
```

Output

```
Tenure_Years
1      21.28
2      19.32
3      18.38
4      18.27
5      18.40
6      18.53
7      21.24
8      17.82
9      16.67
10     15.72
11     17.90
12     17.36
13     17.45
14     16.91
15     17.73
Name: Churn, dtype: float64
```



New customers in their first year show the highest churn rate at 21.28%, and churn generally trends downward as tenure increases, reaching a low of 15.72% at 10 years, before fluctuating again in later years due to smaller sample sizes at high tenure values. This pattern is consistent with the common banking phenomenon of early-tenure attrition, where the first 12–24 months are the most critical window for relationship-building.

7.3.3 Churn Rate by Number of Products

Python — Jupyter Notebook

```
# Calculate churn rate for each number of banking products
product_churn = (
    df.groupby("Num_Products")["Churn"]
      .mean()
)
```

```

    .mul(100)
)

# Display the result
product_churn.round(2)

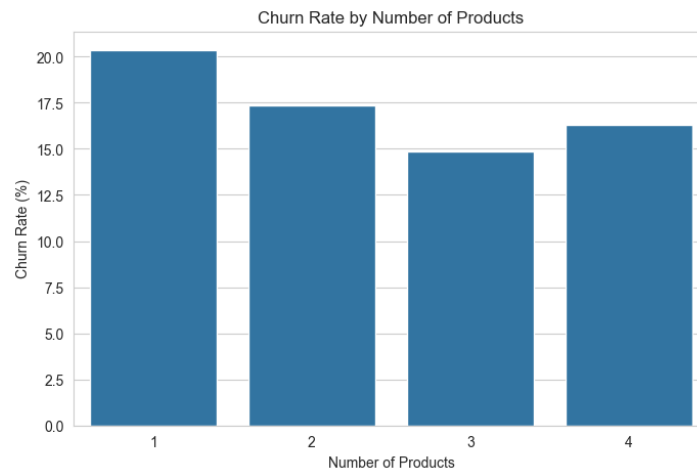
```

Output

```

Num_Products
1    20.35
2    17.37
3    14.86
4    16.29
Name: Churn, dtype: float64

```



Customers holding a single product churn the most (20.35%), and churn falls as the number of products increases — 17.37% at two products and 14.86% at three products — before ticking back up slightly to 16.29% at four products. This is one of the clearer patterns in the dataset: cross-selling additional products appears to be associated with materially lower churn, up to a point.

7.3.4 Churn Rate by Credit Card Ownership

Python — Jupyter Notebook

```

# Calculate churn rate based on credit card ownership
credit_card_churn = (
    df.groupby("Has_Credit_Card")["Churn"]
    .mean()
    .mul(100)
)

# Display the result
credit_card_churn.round(2)

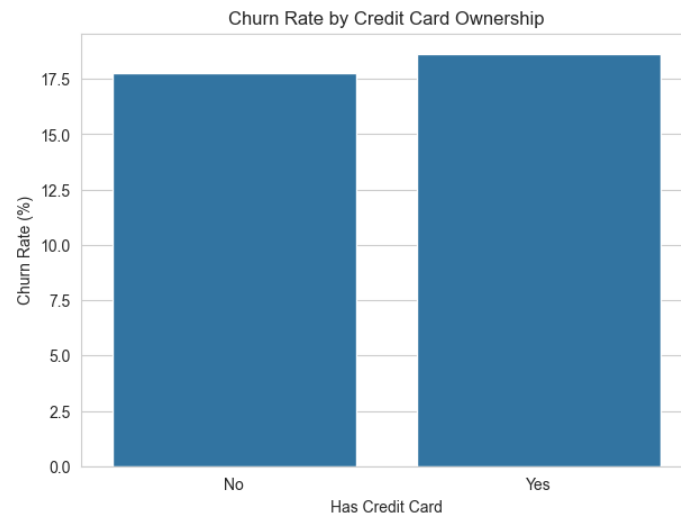
```

Output

```

Has_Credit_Card
No    17.75
Yes   18.63
Name: Churn, dtype: float64

```



Customers who own a credit card churn slightly more (18.63%) than those who do not (17.75%), a small difference that does not, on its own, suggest credit card ownership meaningfully drives retention either way.

7.3.5 Churn Rate by Loan Status

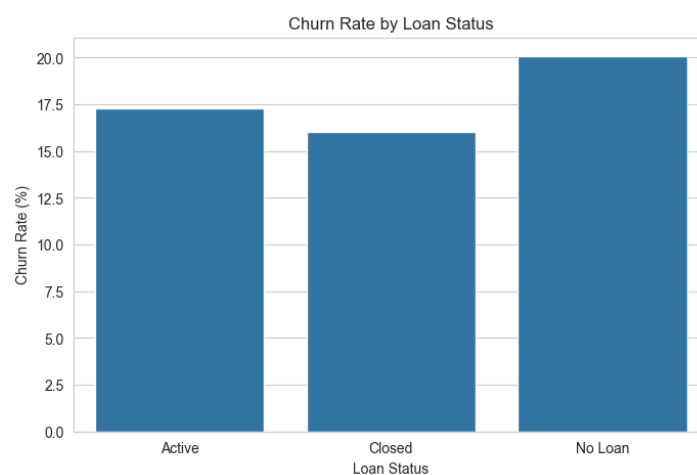
Python — Jupyter Notebook

```
# Calculate churn rate for each loan status
loan_churn = (
    df.groupby("Loan_Status")["Churn"]
      .mean()
      .mul(100)
)

# Display the result
loan_churn.round(2)
```

Output

```
Loan_Status
Active      17.28
Closed      16.03
No Loan     20.07
Name: Churn, dtype: float64
```



Customers with no loan show the highest churn rate (20.07%), while customers with an active loan (17.28%) or a closed loan (16.03%) churn less. This suggests that an ongoing lending relationship — which typically involves more frequent contact and a longer-term financial commitment — is associated with greater customer stickiness.

7.3.6 Churn Rate by Credit Score Range and Group

Python — Jupyter Notebook

```
# Create credit score ranges
df["Credit_Score_Range"] = pd.cut(
    df["Credit_Score"],
    bins=[299, 499, 599, 699, 799, 850],
    labels=[
        "300-499",
        "500-599",
        "600-699",
        "700-799",
        "800-850"
    ]
)

# Calculate churn rate for each credit score range
credit_range_churn = (
    df.groupby("Credit_Score_Range", observed=True)["Churn"]
    .mean()
    .mul(100)
)

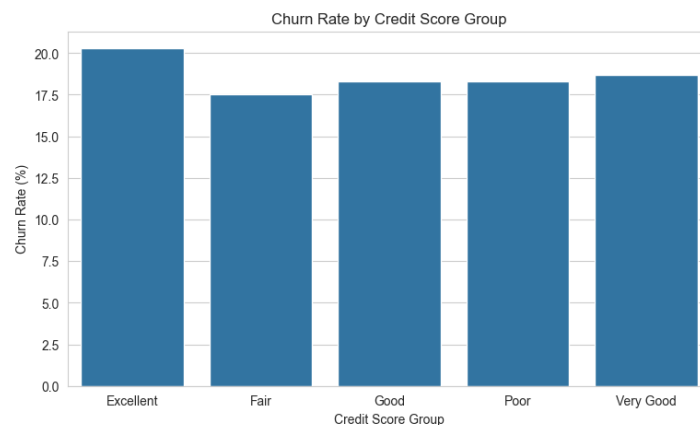
# Display the result
credit_range_churn.round(2)

# Calculate churn rate for each credit score group
credit_score_churn = (
    df.groupby("Credit_Score_Group", observed=True)["Churn"]
    .mean()
    .mul(100)
)

# Display the result
credit_score_churn.round(2)
```

Output

By 100-point range – 500-599: 20.21%, 600-699: 17.22%, 700-799: 18.79%, 800-850: 20.29% (300-499 negligible sample). By 5-tier group – Poor: 18.33%, Fair: 17.52%, Good: 18.30%, Very Good: 18.67%, Excellent: 20.29%.



Both the fine-grained 100-point credit-score ranges and the five-tier Credit_Score_Group show a broadly flat, slightly U-shaped relationship with churn (roughly 17–20% across all bands), with the “Excellent” group (20.29%) and the “Fair” group (17.52%) at the two extremes. Credit score on its own is a weak standalone predictor of churn in this dataset.

7.3.7 Churn Rate by Monthly Income (Range and Category)

Python — Jupyter Notebook

```
# Create income ranges for analysis
df["Income_Range"] = pd.qcut(
    df["Monthly_Income"],
```

```

q=5,
labels=[
    "Very Low",
    "Low",
    "Medium",
    "High",
    "Very High"
]
)

# Calculate churn rate for each income range
income_range_churn = (
    df.groupby("Income_Range", observed=True)["Churn"]
    .mean()
    .mul(100)
)

# Display the result
income_range_churn.round(2)

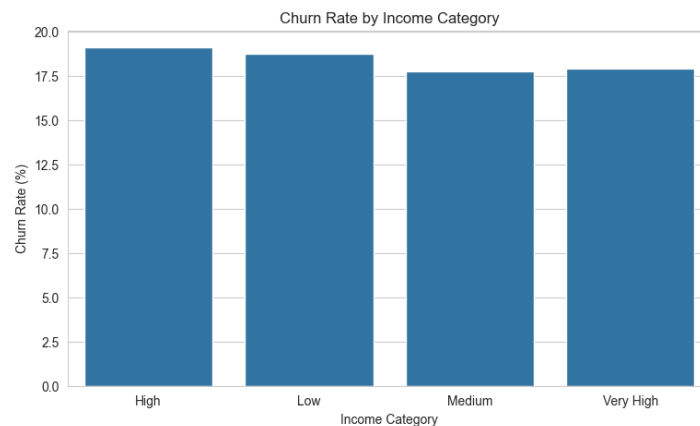
# Calculate churn rate for each income category
income_churn = (
    df.groupby("Income_Category", observed=True)["Churn"]
    .mean()
    .mul(100)
)

# Display the result
income_churn.round(2)

```

Output

By quintile range – Very Low: 19.14%, Low: 17.37%, Medium: 17.50%, High: 19.51%, Very High: 18.38%.
 By quartile category – Low: 18.74%, Medium: 17.76%, High: 19.10%, Very High: 17.91%.



Churn by income shows no strong monotonic trend: the High income category has the highest churn rate (19.10%), while Medium income has the lowest (17.50%). As with credit score, income alone does not appear to be a strong discriminator of churn risk.

7.3.8 Churn Rate by Account Balance Category

Python — Jupyter Notebook

```

# Calculate churn rate for each balance category
balance_churn = (
    df.groupby("Balance_Category", observed=True)["Churn"]
    .mean()
    .mul(100)
)

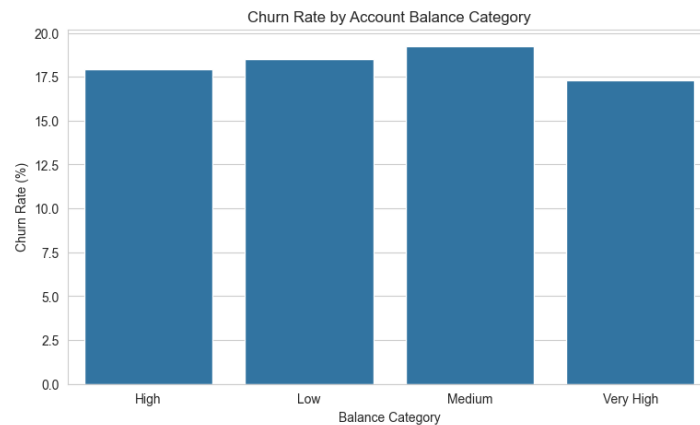
# Display the result
balance_churn.round(2)

```

Output

Balance_Category	Churn Rate (%)
High	17.90
Low	18.48
Medium	19.24

Very High 17.28
Name: Churn, dtype: float64



Churn is fairly stable across balance categories, ranging narrowly from 17.28% (Very High balance) to 19.24% (Medium balance), indicating that account balance alone is not a decisive churn driver.

7.4 Churn by Behavioural Attributes

7.4.1 Churn Rate by Monthly Transactions

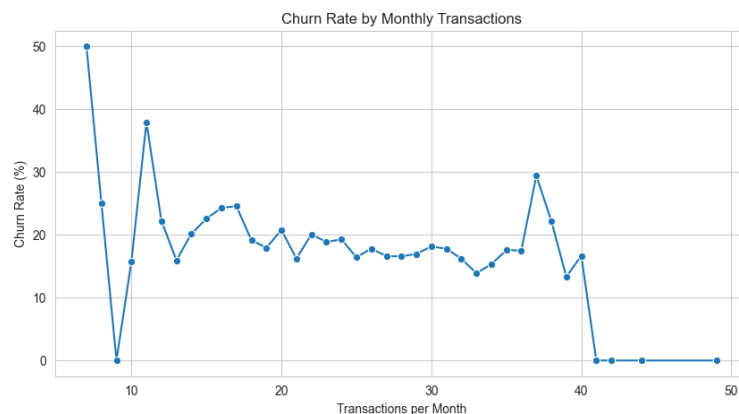
Python — Jupyter Notebook

```
# Calculate churn rate for each transaction level
transaction_churn = (
    df.groupby("Transactions_Per_Month")["Churn"]
      .mean()
      .mul(100)
)

# Display the result
transaction_churn.round(2)
```

Output

```
Transactions_Per_Month
7      50.00
8      25.00
9       0.00
10     15.79
11     37.93
12     22.22
13     15.94
14     20.18
15     22.56
... (output truncated for brevity) ...
42      0.00
44      0.00
49      0.00
Name: Churn, dtype: float64
```



Churn rate by exact transaction count is noisy at the extremes (where very few customers fall), but the bulk of the customer base — roughly 15 to 35 transactions per month — sits in a fairly stable 16–22% churn band, with no strong linear trend. This suggests transaction volume alone is a weak individual predictor, though it remains a useful input alongside other behavioural signals.

7.4.2 Churn Rate by Mobile Banking Usage

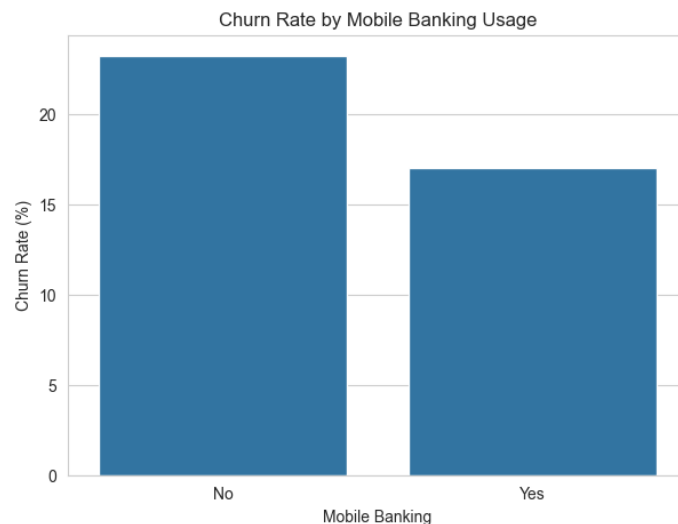
Python — Jupyter Notebook

```
# Calculate churn rate based on mobile banking usage
mobile_churn = (
    df.groupby("Mobile_Banking")["Churn"]
      .mean()
      .mul(100)
)

# Display the result
mobile_churn.round(2)
```

Output

```
Mobile_Banking
No      23.22
Yes     16.99
Name: Churn, dtype: float64
```



This is one of the strongest behavioural signals in the dataset: customers who do not use mobile banking churn at 23.22%, compared to just 16.99% for customers who do. Digital engagement through the mobile app is clearly associated with stronger retention, likely because it deepens the customer's day-to-day relationship with the bank.

7.4.3 Churn Rate by UPI Usage

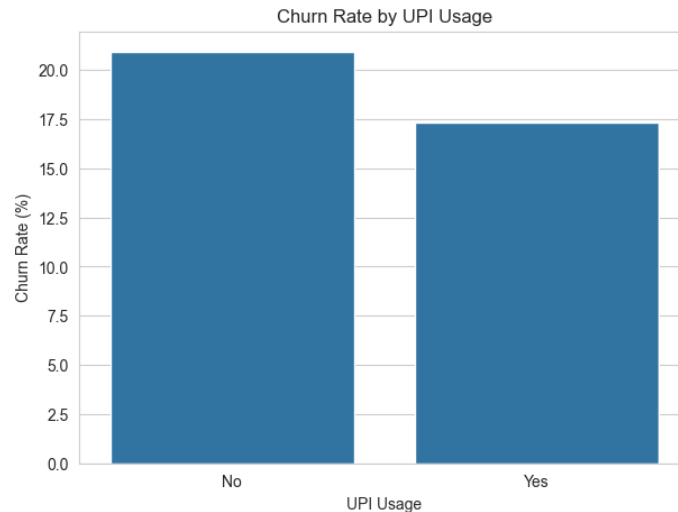
Python — Jupyter Notebook

```
# Calculate churn rate based on UPI usage
upi_churn = (
    df.groupby("UPI_Usage")["Churn"]
      .mean()
      .mul(100)
)

# Display the result
upi_churn.round(2)
```

Output

```
UPI_Usage
No      20.92
Yes     17.34
Name: Churn, dtype: float64
```



A similar, though slightly smaller, effect is seen for UPI usage: non-users churn at 20.92% versus 17.34% for users. Together, mobile banking and UPI usage point to a consistent theme — customers who are digitally active on multiple channels are meaningfully less likely to churn.

7.4.4 Churn Rate by Transaction Recency and Activity Category

Python — Jupyter Notebook

```
# Create ranges for days since the customer's last transaction
df["Transaction_Recency_Group"] = pd.cut(
    df["Days_Since_Last_Transaction"],
    bins=[-1, 30, 60, 90, 180, np.inf],
    labels=[
        "0-30 Days",
        "31-60 Days",
        "61-90 Days",
        "91-180 Days",
        "180+ Days"
    ]
)

# Calculate churn rate for each recency group
recency_churn = (
    df.groupby("Transaction_Recency_Group", observed=True)["Churn"]
    .mean()
    .mul(100)
)

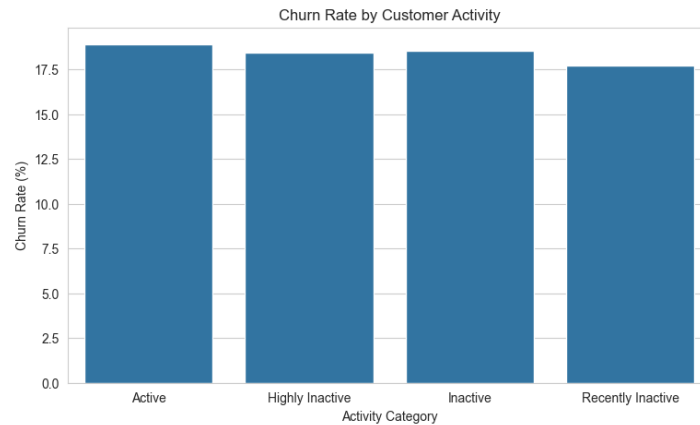
# Display the result
recency_churn.round(2)

# Calculate churn rate for each activity category
activity_churn = (
    df.groupby("Activity_Category", observed=True)["Churn"]
    .mean()
    .mul(100)
)

# Display the result
activity_churn.round(2)
```

Output

```
By day-based recency bucket – 0-30 days: 18.91%, 31-60: 17.40%, 61-90: 18.24%, 91-180: 18.53%, 180+: 18.45%. By Activity_Category – Active: 18.91%, Recently Inactive: 17.71%, Inactive: 18.53%, Highly Inactive: 18.45%.
```



Neither the day-based recency grouping nor the four-tier Activity_Category shows a strong effect on churn (all bands fall within an 17.7–18.9% range). Recency of the last transaction, on its own, is not a strong churn signal in this dataset — which is a useful finding in itself, as it means retention teams should not rely on transaction recency alone as an early-warning indicator.

7.4.5 Churn Rate by Number of Complaints

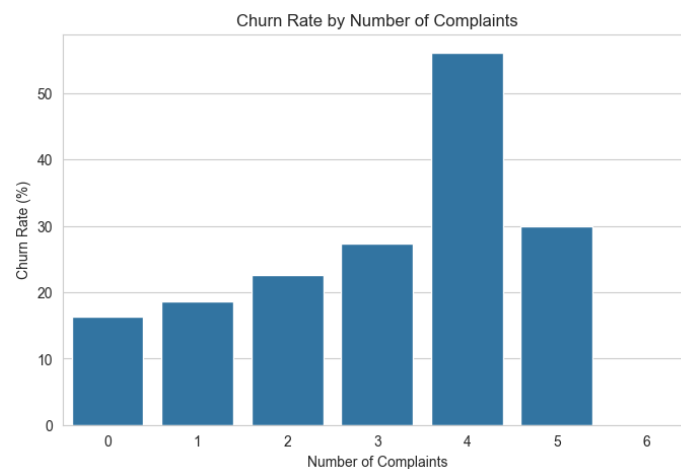
Python — Jupyter Notebook

```
# Calculate churn rate for each complaint level
complaint_churn = (
    df.groupby("Complaints")["Churn"]
      .mean()
      .mul(100)
)

# Display the result
complaint_churn.round(2)
```

Output

```
Complaints
0    16.31
1    18.59
2    22.66
3    27.33
4    56.10
5    30.00
6     0.00
Name: Churn, dtype: float64
```



Complaints show the clearest and most consistent relationship with churn in the entire dataset: churn rises from 16.31% at zero complaints to 18.59% at one complaint, 22.66% at two, 27.33% at three, and peaks at 56.10% for customers with four complaints — more than three times the baseline rate.

This makes number of complaints the single most actionable early-warning indicator identified in this analysis.

7.5 Stayed vs. Churned — Summary Comparison

7.5.1 Average Attribute Values by Churn Status

Python — Jupyter Notebook

```
# COMPARE STAYED VS CHURNED CUSTOMERS

# Select important numerical columns
numeric_columns = [
    "Age",
    "Tenure_Years",
    "Account_Balance",
    "Num_Products",
    "Credit_Score",
    "Monthly_Income",
    "Loan_Amount",
    "Transactions_Per_Month",
    "Complaints"
]

# Calculate the average value of each numerical variable
# for stayed and churned customers
churn_comparison = df.groupby("Churn")[numeric_columns].mean()

# Replace 0 and 1 with meaningful labels
churn_comparison.index = ["Stayed", "Churned"]

# Display the comparison
churn_comparison.round(2)
```

Output

	Age	Tenure_Years	Account_Balance	Num_Products	Credit_Score	\
Stayed	38.05	6.81	129425.79	1.75	709.65	
Churned	38.67	6.58	125609.74	1.65	712.34	

	Monthly_Income	Loan_Amount	Transactions_Per_Month	Complaints
Stayed	55880.99	269111.37	24.10	0.67
Churned	55694.50	235121.87	23.61	0.84

Comparing average values between the two groups reinforces the univariate findings: churned customers hold fewer products on average (1.65 vs 1.75), have a lower average loan amount (₹235,122 vs ₹269,111), and file more complaints on average (0.84 vs 0.67) than customers who stayed. Differences in age, tenure, balance, credit score, income, and transaction frequency between the two groups are all comparatively small.

7.6 Correlation Analysis

7.6.1 Correlation Matrix

A correlation matrix (Pearson) was computed across nine numerical variables — Age, Tenure_Years, Account_Balance, Num_Products, Credit_Score, Monthly_Income, Loan_Amount, Transactions_Per_Month, and Complaints — together with the Churn target, using `df[correlation_columns].corr()`. Rather than reproduce the full 10x10 numeric matrix as text, it is visualised directly as a heatmap below, which makes the pattern of relationships far easier to read.

7.6.2 Correlation Heatmap

Python — Jupyter Notebook

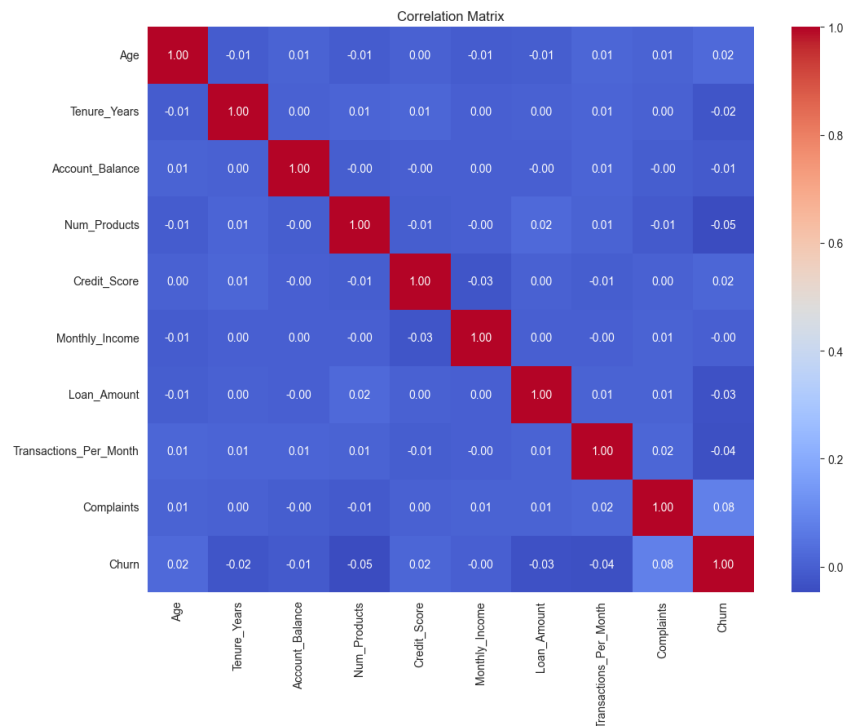
```
# VISUALIZE CORRELATION MATRIX
```

```
# Create the figure
plt.figure(figsize=(12, 9))

# Create a heatmap
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)

# Add title
plt.title("Correlation Matrix")

# Display the chart
plt.show()
```



The heatmap makes it visually clear that no numerical variable in this dataset has a strong linear correlation with Churn — every coefficient lies between -0.05 and $+0.08$. This is expected for a synthetic, well-balanced dataset, and it also carries a genuine analytical lesson: real-world churn is rarely explained by a single variable, and is better understood through the segment-level and categorical patterns explored throughout Section 7, and ultimately through multivariate machine-learning models (see Section 12, Future Improvements).

7.6.3 Variables Most Associated with Churn

Python — Jupyter Notebook

```
# FIND VARIABLES MOST ASSOCIATED WITH CHURN

# Select the correlation values related to Churn
churn_correlation = correlation["Churn"].sort_values(
    ascending=False
)

# Display the correlations
churn_correlation.round(2)
```

Output

```
Churn      1.00
Complaints 0.08
```

```

Age          0.02
Credit_Score 0.02
Monthly_Income -0.00
Account_Balance -0.01
Tenure_Years -0.02
Loan_Amount -0.03
Transactions_Per_Month -0.04
Num_Products -0.05
Name: Churn, dtype: float64

```

Ranked by absolute correlation with Churn, the order is: Complaints (+0.08), Num_Products (−0.05), Transactions_Per_Month (−0.04), Loan_Amount (−0.03), Tenure_Years (−0.02), Age (+0.02) and Credit_Score (+0.02). Complaints is, by a clear margin, the strongest linear correlate of churn among all numerical fields, followed by Num_Products in the opposite (protective) direction.

7.7 EDA Summary

7.7.1 Final Summary Statistics

Python — Jupyter Notebook

```

# FINAL EDA SUMMARY

print("EDA SUMMARY : ")

# Total number of customers
print("Total Customers:", len(df))

# Number of churned customers
print("Churned Customers:", df["Churn"].sum())

# Number of customers who stayed
print("Customers Stayed:", (df["Churn"] == 0).sum())

# Overall churn rate
print("Overall Churn Rate:", round(df["Churn"].mean() * 100, 2), "%")

# Average age
print("Average Age:", round(df["Age"].mean(), 2))

# Average account balance
print("Average Account Balance:", round(df["Account_Balance"].mean(), 2))

# Average credit score
print("Average Credit Score:", round(df["Credit_Score"].mean(), 2))

# Average monthly income
print("Average Monthly Income:", round(df["Monthly_Income"].mean(), 2))

# Average monthly transactions
print(
    "Average Transactions per Month:",
    round(df["Transactions_Per_Month"].mean(), 2)
)

```

Output

```

EDA SUMMARY :
Total Customers: 10000
Churned Customers: 1838
Customers Stayed: 8162
Overall Churn Rate: 18.38 %
Average Age: 38.17
Average Account Balance: 128724.4
Average Credit Score: 710.14
Average Monthly Income: 55846.71
Average Transactions per Month: 24.01

```

The EDA stage confirms a churn rate of 18.38% across 10,000 customers, with an average customer age of 38.17 years, an average account balance of ₹128,724, an average credit score of 710, an average

monthly income of ₹55,847, and an average of 24.01 transactions per month. These summary figures, together with the segment-level findings above, form the evidence base for the business insights presented in Section 11 and for the Power BI dashboard described in Section 8.

7.7.2 Finalising the Dataset for Power BI

Python — Jupyter Notebook

```
# REMOVE TEMPORARY EDA COLUMNS

# Remove temporary columns created only for EDA analysis
columns_to_remove = [
    "Credit_Score_Range",
    "Income_Range",
    "Transaction_Recency_Group"
]

df.drop(columns=columns_to_remove, inplace=True, errors="ignore")

# Display the current columns
df.columns
```

Output

```
Index(['Customer_ID', 'Full_Name', 'Age', 'Gender', 'State', 'City',
      'Account_Type', 'Tenure_Years', 'Account_Balance', 'Num_Products',
      'Has_Credit_Card', 'Loan_Status', 'Credit_Score', 'Monthly_Income',
      'Loan_Amount', 'Transactions_Per_Month', 'Last_Transaction_Date',
      'Mobile_Banking', 'UPI_Usage', 'Complaints', 'Churn', 'Churn_Status',
      'Age_Group', 'Credit_Score_Group', 'Balance_Category',
      'Income_Category', 'Days_Since_Last_Transaction', 'Activity_Category'],
      dtype='str')
```

Python — Jupyter Notebook

```
# CHECK FINAL DATASET STRUCTURE

print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

# Display all columns
df.columns.tolist()
```

Output

```
Number of rows: 10000
Number of columns: 28

['Customer_ID',
 'Full_Name',
 'Age',
 'Gender',
 'State',
 'City',
 'Account_Type',
 'Tenure_Years',
 'Account_Balance',
 'Num_Products',
 'Has_Credit_Card',
 'Loan_Status',
 'Credit_Score',
 'Monthly_Income',
 'Loan_Amount',
 'Transactions_Per_Month',
 'Last_Transaction_Date',
 'Mobile_Banking',
 'UPI_Usage',
 'Complaints',
 'Churn',
 'Churn_Status',
 'Age_Group',
 'Credit_Score_Group',
 'Balance_Category',
 'Income_Category',
 'Days_Since_Last_Transaction',
 'Activity_Category']
```

Python — Jupyter Notebook

```
# EXPORT FINAL CLEANED DATASET
```

```
# Save the final cleaned dataset as a CSV file
df.to_csv("bank_customer_churn_cleaned.csv", index=False)

print("Final cleaned dataset exported successfully.")
```

After removing the temporary, EDA-only helper columns (Credit_Score_Range, Income_Range, and Transaction_Recency_Group) that were created purely to support charting, the final cleaned dataset retains 28 columns — the original 21 plus the seven engineered features described in Section 4.2 — and is exported as `bank_customer_churn_cleaned.csv`. This file is the single data source imported into Power BI for the dashboard described in the next section.

8. Power BI Dashboard

The cleaned, feature-engineered dataset (`bank_customer_churn_cleaned.csv`) was imported into Microsoft Power BI Desktop and used to build a four-page interactive dashboard, Bank Customer Churn Analysis Dashboard. The dashboard uses a consistent magenta/pink colour theme, a persistent left-hand filter panel (State, Gender, Account Type, Age Group, and Loan Status), and a top navigation bar that lets the viewer move between the Overview, Churn Drivers, Customer Segmentation, and Customer Details pages. Screenshots of the actual, published report are included below.

8.1 Overview Page

The Overview page acts as the executive summary of the report. Four KPI cards at the top immediately communicate the headline numbers — 10K total customers, 2K churned customers, an 18.38% churn rate, and 8K retained customers — giving any stakeholder the key figures within seconds of opening the report.

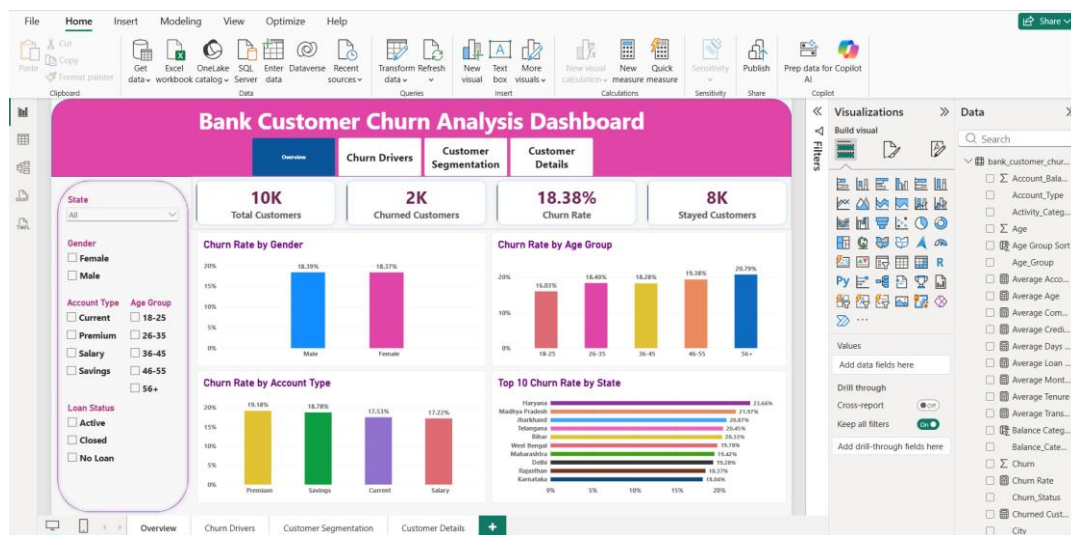


Figure 8.1 — Overview page of the Power BI dashboard, showing headline KPIs and churn breakdowns by gender, age group, account type, and top-10 states.

Below the KPI row, four charts break churn down by Gender, Age Group, Account Type, and the Top 10 states by churn rate — mirroring the corresponding findings from Sections 7.2 and 7.3 of the Python analysis, so that a viewer can move seamlessly between the code-based evidence in this document and the same figures rendered interactively in the dashboard.

8.2 Churn Drivers Page

The Churn Drivers page focuses on the behavioural and service-quality signals identified as most influential during EDA. Four supporting KPI cards show the average credit score (710.14), average transactions per month (24.01), average number of complaints (0.70), and the average number of days since the last transaction (53.37).

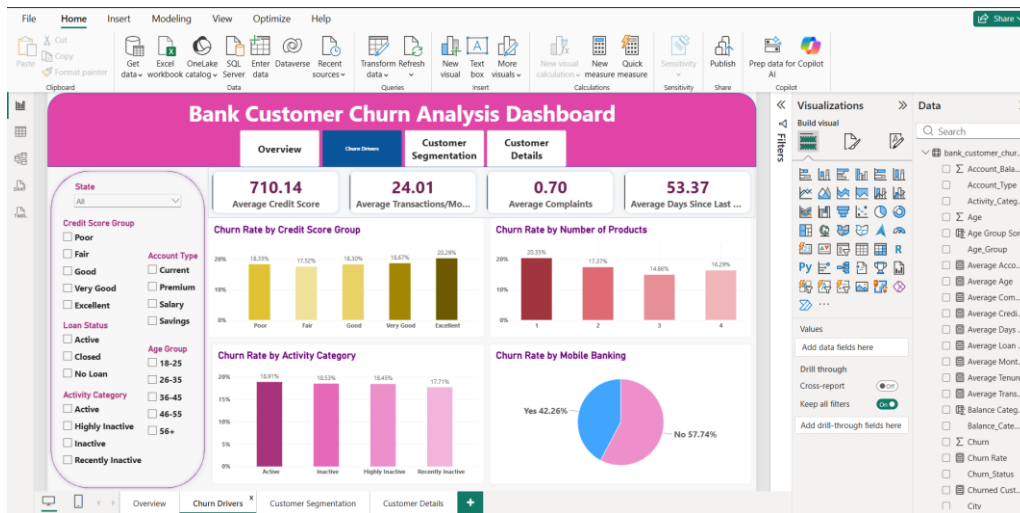


Figure 8.2 — Churn Drivers page, breaking churn down by credit score group, number of products, activity category, and mobile banking usage.

The four charts on this page — Churn Rate by Credit Score Group, Churn Rate by Number of Products, Churn Rate by Activity Category, and Churn Rate by Mobile Banking — give the retention team a focused, one-screen view of the levers most associated with churn, consistent with the correlation findings in Section 7.6.

8.3 Customer Segmentation Page

The Customer Segmentation page is designed for portfolio-level analysis. Four KPI cards summarise the average account balance (₹128.72K), average monthly income (₹55.85K), average tenure (6.77 years), and average loan amount (₹262.86K) across the filtered customer base.

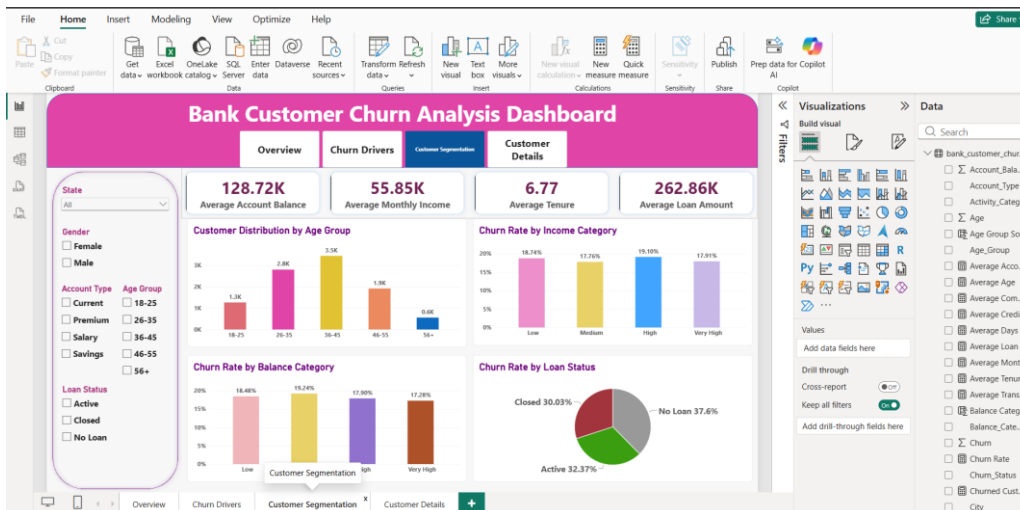


Figure 8.3 — Customer Segmentation page, showing customer distribution by age group and churn broken down by income category, balance category, and loan status.

The charts on this page — Customer Distribution by Age Group, Churn Rate by Income Category, Churn Rate by Balance Category, and Churn Rate by Loan Status — let a business user quickly identify which financial segments carry the most churn risk and, combined with the segment sizes, prioritise where a retention campaign would have the greatest impact.

8.4 Customer Details Page

The Customer Details page provides a searchable, single-customer drill-down view. A dropdown selector (bound to Customer_ID) lets a relationship manager pull up any individual customer's full

profile, organised into three clearly labelled panels — Personal Information, Banking & Financial Information, and Customer Activity & Churn Information.

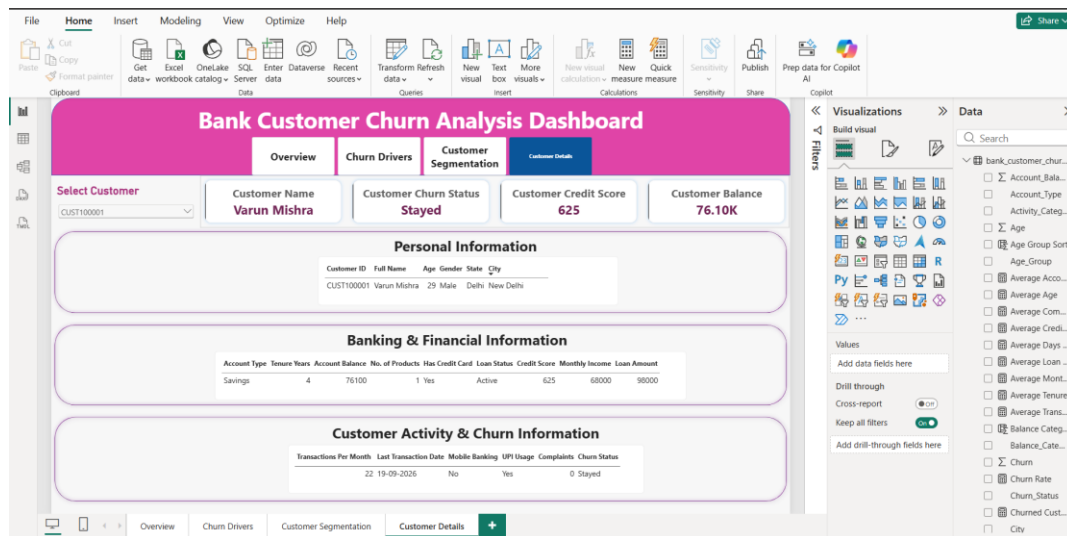


Figure 8.4 — Customer Details page, showing the full profile of an individual customer (CUST100001, Varun Mishra), including personal, banking, and activity information.

This page turns the aggregate, dashboard-level insights from the previous three pages into a practical, front-line tool: a bank staff member preparing for a customer conversation can look up the customer's churn status, credit score, account balance, loan details, transaction activity, and complaint history in one place before making contact.

8.5 Dashboard Design Notes

- A single, consistent colour theme (magenta/pink header bar with complementary chart palettes) is used across all four pages for visual coherence.
- The left-hand filter panel (State, Gender, Account Type, Age Group, Loan Status, and, on later pages, Credit Score Group and Activity Category) is present on every page and applies consistently across all visuals, enabling true cross-filtered exploration.
- KPI cards are placed at the top of every page so that headline numbers are always visible before the viewer scrolls into the supporting charts.
- The Customer Details page uses a searchable slicer rather than a static table, making the dashboard usable for individual customer look-ups rather than only aggregate analysis.

9. Business Insights

Synthesising the Python-based EDA (Section 7) and the Power BI dashboard (Section 8), the following table summarises the most actionable business insights uncovered during this analysis, ranked roughly by the strength of the effect observed.

Driver	Type of Effect	Key Finding	Recommended Action
Complaints	Strongest driver identified	Churn rises from 16.31% (0 complaints) to 56.10% (4 complaints).	Route customers with 2+ complaints to a proactive service-recovery workflow before they reach 3–4 complaints.

Driver	Type of Effect	Key Finding	Recommended Action
Number of Products	Strong protective factor	Churn falls from 20.35% (1 product) to 14.86% (3 products).	Prioritise cross-sell of a second and third product for single-product customers, especially in year one.
Mobile Banking / UPI Usage	Strong protective factor	Non-users churn at 23.22% / 20.92% vs 16.99% / 17.34% for users.	Run digital-adoption campaigns (app onboarding, UPI activation incentives) targeted at non-digital customers.
Tenure (Year 1)	Early-tenure risk	First-year customers churn at 21.28%, the highest of all tenure bands.	Strengthen the first 12-month onboarding journey with structured check-ins and welcome offers.
Account Type (Premium)	Elevated-value segment at risk	Premium accounts churn at 19.18%, the highest of the four account types.	Assign relationship managers to monitor and engage high-value Premium customers proactively.
Geography (State & City)	Regional concentration	Haryana (23.66%) and cities such as Gurugram (26.56%) and Jabalpur (26.09%) show the highest churn.	Focus regional retention resources and root-cause review on the highest-churn branches/cities first.
Age (56+)	Mild demographic risk	Churn rises gradually from 16.03% (18-25) to 20.79% (56+).	Tailor retention offers for senior customers around service convenience and in-branch support.
Loan Status (No Loan)	Moderate risk	“No Loan” customers churn at 20.07% vs 16-17% for customers with a loan.	Explore small-ticket credit or loan products to deepen the relationship with no-loan customers.

9.1 Overall Takeaway

No single variable in this dataset drives churn in isolation — every individual correlation coefficient with Churn falls between -0.05 and +0.08. This means the most effective retention strategy is unlikely to rely on any one signal, but should instead combine service-quality monitoring (complaints), product depth (number of products held), digital engagement (mobile banking / UPI), and early-tenure relationship management into a single, layered retention programme, supported by the geographic and demographic targeting made possible by the Power BI dashboard.

10. Challenges Faced

- Weak individual correlations: with every numerical variable correlating with Churn at below 0.10, no single “silver bullet” predictor emerged from the correlation analysis, requiring a broader, segment-by-segment exploration (Section 7) rather than reliance on a single statistic.
- Choosing meaningful bin boundaries: deciding where to place the cut points for Age_Group, Credit_Score_Group, Balance_Category, and Activity_Category required balancing standard industry conventions (e.g. widely used credit-score bands) against the actual distribution of this dataset.

- Large categorical cardinality: the City column contains 90 distinct values, which is too many to visualise directly; a Top-10 ranking approach was used instead to keep the state/city-level charts readable.
- Consistent text formatting: raw categorical fields required careful trimming and case-standardisation (Section 6.5) to avoid the same category being silently split into multiple values (e.g. “yes”, “Yes”, “ Yes”) during grouping and aggregation.
- Designing a multi-page Power BI report: distributing 28 columns and dozens of possible cuts across just four dashboard pages required deciding which visuals belonged on an executive Overview versus a deeper Churn Drivers or Segmentation page, without overwhelming any single page.
- Keeping cross-page filters consistent: ensuring the same slicer panel (State, Gender, Account Type, Age Group, Loan Status) behaved consistently and applied correctly to every visual across all four report pages.

11. Future Improvements

This project focused on descriptive and diagnostic analytics — understanding what happened and why. The following extensions would move the project toward predictive and prescriptive analytics:

- Predictive modelling: train classification models (Logistic Regression, Random Forest, Gradient Boosting / XGBoost) on the engineered feature set to produce a churn-probability score for every customer, rather than relying on aggregate segment rates.
- Feature importance and SHAP analysis: use model-based feature-importance techniques to validate and extend the correlation-based ranking in Section 7.6 with a fully multivariate view.
- Customer Lifetime Value (CLV): combine churn probability with account balance, product holding, and tenure to estimate CLV and prioritise retention spend on high-value, high-risk customers.
- Time-series / cohort analysis: incorporate transaction-level history (rather than a single snapshot) to study how churn risk evolves over a customer's lifecycle and to build early-warning trend indicators.
- Automated data refresh: connect Power BI to a live database or scheduled data pipeline (Power Query dataflows) so the dashboard updates automatically as new transactions and complaints are recorded.
- What-if and scenario analysis: add Power BI what-if parameters (e.g. “what if complaint resolution time improves by X%”) to let business users simulate the impact of retention initiatives directly in the dashboard.
- Model deployment: expose the trained churn model as a scored table or API, and surface the resulting risk score directly on the Customer Details page for front-line staff.

12. Conclusion

This project delivered a complete, end-to-end churn analytics workflow for a simulated bank customer base of 10,000 records. Starting from a raw, 21-column CSV file, a rigorous Python pipeline was used to inspect, clean, and validate the data, followed by the engineering of seven new analytical features. A structured exploratory data analysis then quantified how churn — which stands at an overall rate of 18.38% — varies across every demographic, financial, account, and behavioural dimension in the dataset, with the number of customer complaints emerging as the strongest single driver and the number of banking products, mobile banking usage, and UPI usage emerging as the clearest protective factors.

These findings were then translated into a four-page, interactive Power BI dashboard covering an executive overview, a focused churn-drivers view, a customer-segmentation view, and a searchable individual customer profile — giving stakeholders at every level, from senior management to front-line relationship managers, a way to explore and act on the same underlying evidence. Together, the Python notebook and the Power BI report demonstrate a realistic, professional analytics workflow suitable for a data analyst portfolio, and this document has aimed to make every step of that workflow — code, output, chart, and dashboard — fully transparent and reproducible.

13. References

- Pandas Documentation — <https://pandas.pydata.org/docs/>
- NumPy Documentation — <https://numpy.org/doc/>
- Matplotlib Documentation — <https://matplotlib.org/stable/contents.html>
- Seaborn Documentation — <https://seaborn.pydata.org/>
- Jupyter Project Documentation — <https://docs.jupyter.org/>
- Microsoft Power BI Documentation — <https://learn.microsoft.com/en-us/power-bi/>
- Project source files: Bank_Customer_Churn_Analysis.ipynb and Bank_Customer_Churn_Analysis_Dashboard.pbix